

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №7

Специальность ПО-13

Выполнил:
М.А. Романюк
студент группы ПО-13
Проверил:
А.А.Крощенко
доц. кафедры ИИТ,
14.05.2026

Брест 2026

Цель работы: освоить возможности языка программирования Python в разработке оконных приложений

Общее задание:

Задание 1:

Построение графических примитивов и надписей

Требования к выполнению

- Реализовать соответствующие классы, указанные в задании;
- Организовать ввод параметров для создания объектов (использовать экранные компоненты);
- Осуществить визуализацию графических примитивов

Вариант 9: Создать классы Point и Line. Объявить список из n объектов класса Point.

Для объекта класса Line определить, какие из объектов Point лежат на одной стороне от прямой линии и какие на другой. Реализовать ввод данных для объекта Line и случайное задание данных для объекта Point.

Задание 2.

Реализовать построение заданного типа фрактала по варианту

Везде, где это необходимо, предусмотреть ввод параметров, влияющих на внешний вид фрактала

Вариант 9: Остров Минковского

Код программы

Task1.py

```
import tkinter as tk
from tkinter import ttk, messagebox
import random
import math
from datetime import datetime

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```

def side_of_line(self, line):
    x1, y1 = line.p1.x, line.p1.y
    x2, y2 = line.p2.x, line.p2.y
    value = (x2 - x1) * (self.y - y1) - (y2 - y1) * (self.x - x1)
    if value > 0:
        return 1
    if value < 0:
        return -1
    return 0

def get_coordinates(self):
    return (self.x, self.y)

class Line:
    def __init__(self, p1, p2):
        self.p1 = p1
        self.p2 = p2

    def is_near(self, point, threshold=20):
        x1, y1 = self.p1.x, self.p1.y
        x2, y2 = self.p2.x, self.p2.y
        x0, y0 = point.x, point.y
        dist = abs((y2 - y1) * x0 - (x2 - x1) * y0 + x2 * y1 - y2 * x1)
        dist = dist / math.hypot(y2 - y1, x2 - x1)
        return dist <= threshold

    def length(self):
        return math.hypot(self.p2.x - self.p1.x, self.p2.y - self.p1.y)

class PointSideCalculator:
    @staticmethod
    def get_side_color(point, line):
        if not line:
            return "black"
        side = point.side_of_line(line)
        if side == 1:
            return "green"
        if side == -1:
            return "blue"
        return "orange"

    @staticmethod
    def classify_points(points, line):
        side1, side2, on_line = [], [], []
        for point in points:
            coord_str = f"({point.x},{point.y})"

```

```

        side = point.side_of_line(line)
        if side == 1:
            side1.append(coord_str)
        elif side == -1:
            side2.append(coord_str)
        else:
            on_line.append(coord_str)
    return side1, side2, on_line

```

```

class LineDrawer:

```

```

    @staticmethod

```

```

    def draw_line_with_labels(canvas, line):

```

```

        if not line:

```

```

            return

```

```

        x1, y1 = line.p1.x, line.p1.y

```

```

        x2, y2 = line.p2.x, line.p2.y

```

```

        canvas.create_line(x1, y1, x2, y2, fill="red", width=2)

```

```

        mx = (x1 + x2) / 2

```

```

        my = (y1 + y2) / 2

```

```

        dx, dy = x2 - x1, y2 - y1

```

```

        nx, ny = -dy, dx

```

```

        length = math.hypot(nx, ny)

```

```

        if length != 0:

```

```

            nx, ny = nx / length * 20, ny / length * 20

```

```

            canvas.create_text(mx + nx, my + ny, text="Side +1", fill="green")

```

```

            canvas.create_text(mx - nx, my - ny, text="Side -1", fill="blue")

```

```

    @staticmethod

```

```

    def draw_points(canvas, points, line):

```

```

        for i, point in enumerate(points):

```

```

            color = PointSideCalculator.get_side_color(point, line)

```

```

            canvas.create_oval(point.x - 4, point.y - 4, point.x + 4, point.y + 4,

```

```

fill=color, outline="black")

```

```

            canvas.create_text(point.x + 8, point.y - 8, text=str(i + 1), font=("Arial",

```

```

8))

```

```

class AnimationManager:

```

```

    def __init__(self, canvas, points, get_speed):

```

```

        self.canvas = canvas

```

```

        self.points = points

```

```

        self.get_speed = get_speed

```

```

        self.running = False

```

```

        self.index = 0

```

```

    def start(self):

```

```

        self.running = True

```

```

        self.index = 0
        self._animate()

    def stop(self):
        self.running = False

    def _animate(self):
        if not self.running:
            return
        if self.index < len(self.points):
            point = self.points[self.index]
            self.canvas.create_oval(
                point.x - 6, point.y - 6, point.x + 6, point.y + 6, fill="red",
outline="black", tags=f"h_{self.index}"
            )
            self.canvas.after(100, self._remove_highlight, self.index)
            self.index += 1
            self.canvas.after(self.get_speed(), self._animate)
        else:
            self.running = False

    def _remove_highlight(self, index):
        self.canvas.delete(f"h_{index}")

class UIController:
    """Класс для управления UI элементами, чтобы уменьшить количество атрибутов в
    PointVisualizer"""

    def __init__(self, parent):
        self.parent = parent
        self.vars = {
            "points_count": tk.IntVar(value=10),
            "line_x1": tk.IntVar(value=200),
            "line_y1": tk.IntVar(value=100),
            "line_x2": tk.IntVar(value=600),
            "line_y2": tk.IntVar(value=500),
            "speed_var": tk.IntVar(value=50),
        }
        self.widgets = {}

    def setup_ui(self, window):
        control_frame = ttk.Frame(window, padding="10")
        control_frame.pack(side=tk.TOP, fill=tk.X)

        # Панель точек
        ttk.Label(control_frame, text="Number of points:").grid(row=0, column=0, padx=5)

```

```

        ttk.Spinbox(control_frame, from_=1, to=50,
textvariable=self.vars["points_count"], width=10).grid(
            row=0, column=1, padx=5
        )
        ttk.Button(control_frame, text="Generate Random Points",
command=self.parent.generate_random_points).grid(
            row=0, column=2, padx=5
        )
        ttk.Button(control_frame, text="Clear Points",
command=self.parent.clear_points).grid(row=0, column=3, padx=5)

        ttk.Separator(control_frame, orient="vertical").grid(row=0, column=4, rowspan=3,
padx=10, sticky="ns")

        # Панель линии
        ttk.Label(control_frame, text="Line Point 1 (x,y):").grid(row=0, column=5,
padx=5)
        ttk.Entry(control_frame, textvariable=self.vars["line_x1"], width=5).grid(row=0,
column=6)
        ttk.Entry(control_frame, textvariable=self.vars["line_y1"], width=5).grid(row=0,
column=7)

        ttk.Label(control_frame, text="Line Point 2 (x,y):").grid(row=1, column=5,
padx=5)
        ttk.Entry(control_frame, textvariable=self.vars["line_x2"], width=5).grid(row=1,
column=6)
        ttk.Entry(control_frame, textvariable=self.vars["line_y2"], width=5).grid(row=1,
column=7)

        ttk.Button(control_frame, text="Draw Line", command=self.parent.draw_line).grid(
            row=0, column=8, rowspan=2, padx=5
        )
        ttk.Button(control_frame, text="Analyze Points",
command=self.parent.analyze_points).grid(
            row=0, column=9, rowspan=2, padx=5
        )

        ttk.Separator(control_frame, orient="vertical").grid(row=0, column=10,
rowspan=3, padx=10, sticky="ns")

        # Панель анимации
        ttk.Label(control_frame, text="Animation Speed (ms):").grid(row=0, column=11,
padx=5)
        ttk.Scale(
            control_frame,
            from_=10,
            to=500,
            variable=self.vars["speed_var"],

```

```

        orient=tk.HORIZONTAL,
        length=100,
        command=self._update_speed,
    ).grid(row=0, column=12)
    self.widgets["speed_label"] = ttk.Label(control_frame, text="50ms")
    self.widgets["speed_label"].grid(row=0, column=13)

    self.widgets["animate_btn"] = ttk.Button(
        control_frame, text="Start Animation", command=self.parent.toggle_animation
    )
    self.widgets["animate_btn"].grid(row=1, column=11, columnspan=3)

    ttk.Button(control_frame, text="Screenshot",
command=self.parent.take_screenshot).grid(
        row=2, column=0, columnspan=2
    )
    ttk.Button(control_frame, text="Exit",
command=self.parent.on_closing).grid(row=2, column=2, columnspan=2)

    # Текстовое поле для результатов
    self.widgets["result_text"] = tk.Text(window, height=8, width=80)
    self.widgets["result_text"].pack(side=tk.BOTTOM, fill=tk.X, padx=10, pady=5)

    # Холст для рисования
    canvas = tk.Canvas(window, bg="white", width=800, height=500)
    canvas.pack(side=tk.TOP, fill=tk.BOTH, expand=True, padx=10, pady=10)

    return canvas

def _update_speed(self, val):
    self.parent.speed = int(float(val))
    if "speed_label" in self.widgets:
        self.widgets["speed_label"].config(text=f"{self.parent.speed}ms")

def update_animation_button(self, is_running):
    if "animate_btn" in self.widgets:
        self.widgets["animate_btn"].config(text="Stop Animation" if is_running else
"Start Animation")

def display_results(self, side1, side2, on_line):
    result_text = self.widgets["result_text"]
    result_text.delete(1.0, tk.END)
    result_text.insert(tk.END, f"Points on side +1: {len(side1)}\n")
    result_text.insert(tk.END, f"  {'', '.join(side1) if side1 else 'None'}\n\n")
    result_text.insert(tk.END, f"Points on side -1: {len(side2)}\n")
    result_text.insert(tk.END, f"  {'', '.join(side2) if side2 else 'None'}\n\n")
    result_text.insert(tk.END, f"Points on line: {len(on_line)}\n")
    result_text.insert(tk.END, f"  {'', '.join(on_line) if on_line else 'None'}")

```

```

class PointVisualizer:
    """Теперь здесь ТОЛЬКО 6 атрибутов!"""

    def __init__(self, window):
        self.window = window # 1
        self.points = [] # 2
        self.line = None # 3
        self.speed = 50 # 4
        self.anim_manager = None # 5
        self.ui = UIController(self) # 6 - ВСЕ UI элементы в одном объекте!

        self._setup()

    def _setup(self):
        self.window.title("Point and Line Visualizer")
        self.window.geometry("900x700")

        self.canvas = self.ui.setup_ui(self.window)
        self.canvas.bind("<Button-1>", self.on_canvas_click)
        self.window.protocol("WM_DELETE_WINDOW", self.on_closing)

    def _get_current_speed(self):
        return self.speed

    def generate_random_points(self):
        self.clear_points()
        count = self.ui.vars["points_count"].get()
        for _ in range(count):
            x = random.randint(50, 750)
            y = random.randint(50, 450)
            self.points.append(Point(x, y))
        self.draw_all()

    def clear_points(self):
        self.points = []
        self.draw_all()

    def draw_line(self):
        x1, y1 = self.ui.vars["line_x1"].get(), self.ui.vars["line_y1"].get()
        x2, y2 = self.ui.vars["line_x2"].get(), self.ui.vars["line_y2"].get()
        self.line = Line(Point(x1, y1), Point(x2, y2))
        self.draw_all()

    def draw_all(self):
        self.canvas.delete("all")
        LineDrawer.draw_line_with_labels(self.canvas, self.line)

```

```

        LineDrawer.draw_points(self.canvas, self.points, self.line)

def on_canvas_click(self, event):
    self.points.append(Point(event.x, event.y))
    self.draw_all()

def analyze_points(self):
    if not self.line:
        messagebox.showwarning("Warning", "Draw a line first!")
        return
    side1, side2, on_line = PointSideCalculator.classify_points(self.points,
self.line)
    self.ui.display_results(side1, side2, on_line)
    self.draw_all()

def toggle_animation(self):
    if not self.anim_manager:
        self.anim_manager = AnimationManager(self.canvas, self.points,
self._get_current_speed)
    if not self.anim_manager.running:
        self.anim_manager.start()
        self.ui.update_animation_button(True)
    else:
        self.anim_manager.stop()
        self.ui.update_animation_button(False)

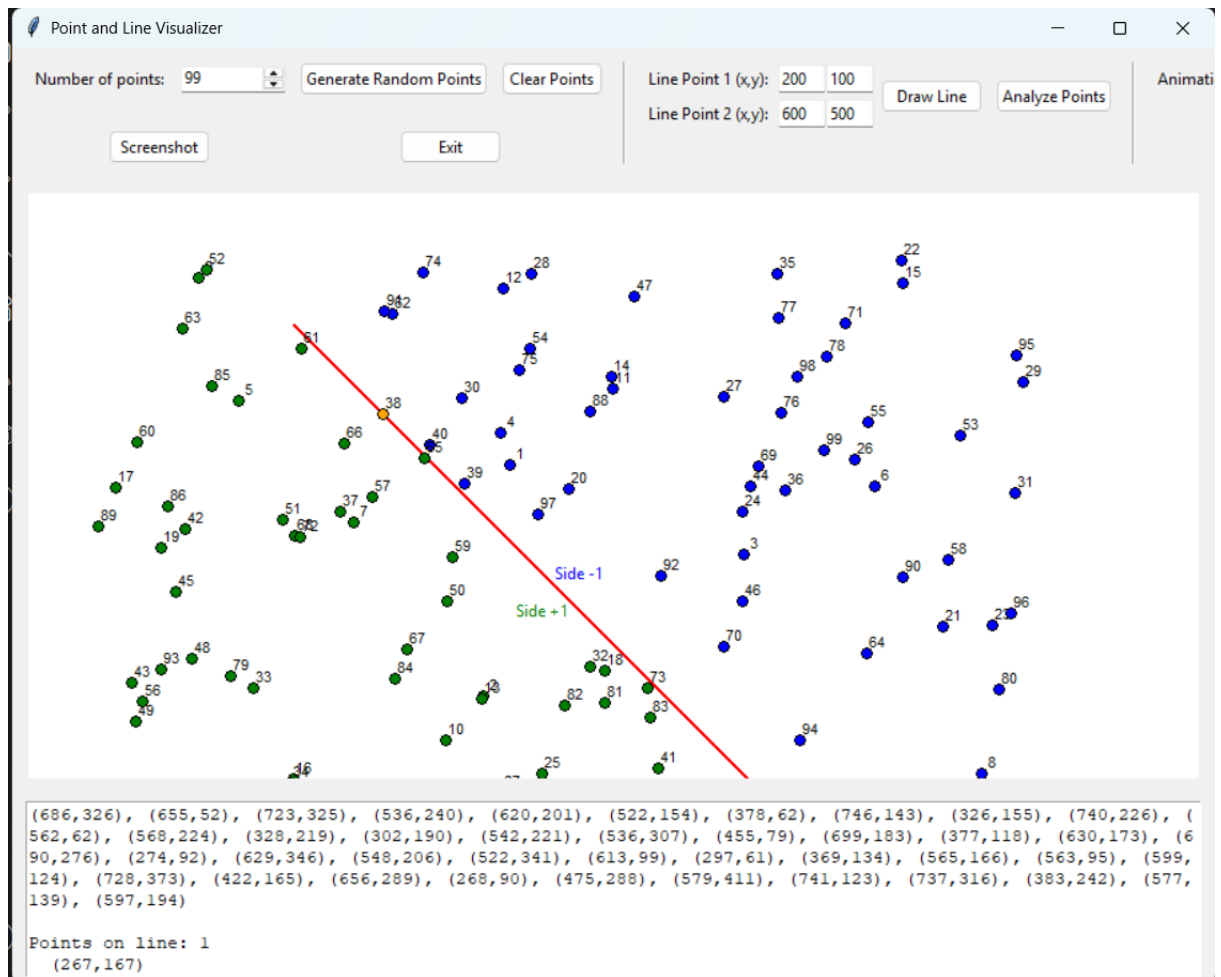
def take_screenshot(self):
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    filename = f"screenshot_{timestamp}.ps"
    self.canvas.postscript(file=filename, colormode="color")
    messagebox.showinfo("Screenshot", f"Screenshot saved as {filename}")

def on_closing(self):
    if self.anim_manager:
        self.anim_manager.stop()
    self.window.quit()
    self.window.destroy()

if __name__ == "__main__":
    app = tk.Tk()
    PointVisualizer(app)
    app.mainloop()

```

Рисунки с выполнением программы



Task2.py

```
import turtle
import tkinter as tk
from tkinter import ttk, colorchooser
import math
from dataclasses import dataclass
```

```
@dataclass
class FractalParams:
    level: tk.IntVar = None
    length: tk.IntVar = None
    speed: tk.IntVar = None
    color: str = "#FF0000"
    bg_color: str = "#FFFFFF"
    pen_size: tk.IntVar = None

    def __post_init__(self):
        if self.level is None:
```

```

        self.level = tk.IntVar(value=1)
    if self.length is None:
        self.length = tk.IntVar(value=150)
    if self.speed is None:
        self.speed = tk.IntVar(value=5)
    if self.pen_size is None:
        self.pen_size = tk.IntVar(value=2)

    def get_all_params(self):
        return (self.level.get(), self.length.get(), self.speed.get(),
                self.color, self.bg_color, self.pen_size.get())

class UIControls:
    def __init__(self, parent, params):
        self.parent = parent
        self.params = params
        self.level_label = None
        self.length_label = None
        self.speed_label = None
        self.color_preview = None

    def create_level_control(self):
        ttk.Label(self.parent, text="Recursion level (0-4):").pack(anchor=tk.W)
        level_frame = ttk.Frame(self.parent)
        level_frame.pack(fill=tk.X, pady=(0, 10))
        level_scale = ttk.Scale(
            level_frame, from_=0, to=4, variable=self.params.level,
command=self._update_level_label
        )
        level_scale.pack(side=tk.LEFT, fill=tk.X, expand=True)
        self.level_label = ttk.Label(level_frame, text="1", width=5, font=("Arial", 10,
"bold"))
        self.level_label.pack(side=tk.RIGHT, padx=(5, 0))

    def create_length_control(self):
        ttk.Label(self.parent, text="Side length (50-200):").pack(anchor=tk.W)
        length_frame = ttk.Frame(self.parent)
        length_frame.pack(fill=tk.X, pady=(0, 10))
        length_scale = ttk.Scale(
            length_frame, from_=50, to=200, variable=self.params.length,
command=self._update_length_label
        )
        length_scale.pack(side=tk.LEFT, fill=tk.X, expand=True)
        self.length_label = ttk.Label(length_frame, text="150", width=5, font=("Arial",
10, "bold"))
        self.length_label.pack(side=tk.RIGHT, padx=(5, 0))

```

```

def create_speed_control(self):
    ttk.Label(self.parent, text="Speed (0-10):").pack(anchor=tk.W)
    speed_frame = ttk.Frame(self.parent)
    speed_frame.pack(fill=tk.X, pady=(0, 10))
    speed_scale = ttk.Scale(
        speed_frame, from_=0, to=10, variable=self.params.speed,
command=self._update_speed_label
    )
    speed_scale.pack(side=tk.LEFT, fill=tk.X, expand=True)
    self.speed_label = ttk.Label(speed_frame, text="5", width=5)
    self.speed_label.pack(side=tk.RIGHT, padx=(5, 0))

def create_pen_control(self):
    ttk.Label(self.parent, text="Pen size (1-5):").pack(anchor=tk.W)
    pen_frame = ttk.Frame(self.parent)
    pen_frame.pack(fill=tk.X, pady=(0, 10))
    pen_scale = ttk.Scale(pen_frame, from_=1, to=5, variable=self.params.pen_size)
    pen_scale.pack(side=tk.LEFT, fill=tk.X, expand=True)

def create_color_controls(self, color_callback, bg_callback):
    ttk.Label(self.parent, text="Colors:").pack(anchor=tk.W, pady=(5, 0))
    color_frame = ttk.Frame(self.parent)
    color_frame.pack(fill=tk.X, pady=5)

    ttk.Button(color_frame, text="Line color",
command=color_callback).pack(side=tk.LEFT, padx=(0, 5))

    ttk.Button(color_frame, text="Background color",
command=bg_callback).pack(side=tk.LEFT)

    self.color_preview = tk.Canvas(color_frame, width=30, height=25,
bg=self.params.color, highlightthickness=1)
    self.color_preview.pack(side=tk.RIGHT, padx=5)

def _update_level_label(self, _=None):
    level = int(self.params.level.get())
    self.level_label.config(text=str(level))

def _update_length_label(self, _=None):
    self.length_label.config(text=str(int(self.params.length.get())))

def _update_speed_label(self, _=None):
    self.speed_label.config(text=str(int(self.params.speed.get())))

class FractalRenderer:
    def __init__(self, params):
        self.params = params

```

```

self.turtle_window = None
self.canvas = None
self.turtle_obj = None
self.is_drawing = False

def close_window(self):
    if self.turtle_window and self.turtle_window.winfo_exists():
        self.is_drawing = False
        self.turtle_window.destroy()
        self.turtle_window = None
        self.canvas = None
        self.turtle_obj = None

def test_drawing(self, root, progress_callback):
    self.close_window()

    self.turtle_window = tk.Toplevel(root)
    self.turtle_window.title("Test Drawing")
    self.turtle_window.geometry("600x600")
    self.turtle_window.protocol("WM_DELETE_WINDOW", self.close_window)

    self.canvas = tk.Canvas(self.turtle_window, bg="white", highlightthickness=1)
    self.canvas.pack(fill=tk.BOTH, expand=True)

    self.turtle_window.update()

    self.turtle_obj = turtle.RawTurtle(self.canvas)

    canvas_width = self.canvas.winfo_width()
    canvas_height = self.canvas.winfo_height()

    self.turtle_obj.screen.setworldcoordinates(0, canvas_height, canvas_width, 0)

    self.turtle_obj.speed(3)
    self.turtle_obj.pencolor("red")
    self.turtle_obj.pensize(3)
    self.turtle_obj.penup()

    square_size = 100
    start_x = (canvas_width - square_size) / 2
    start_y = (canvas_height - square_size) / 2

    self.turtle_obj.goto(start_x, start_y)
    self.turtle_obj.pendown()

    for _ in range(4):
        self.turtle_obj.forward(square_size)
        self.turtle_obj.right(90)

```

```

self.turtle_obj.hideturtle()
progress_callback("Test square drawn")

self.canvas.create_text(
    canvas_width // 2, 30, text="If you see a red square - drawing works",
    fill="blue", font=("Arial", 10)
)

def _draw_segment(self, length, level):
    if level == 0 or not self.is_drawing:
        if self.is_drawing and self.turtle_obj:
            self.turtle_obj.forward(length)
        return

    new_len = length / 4.0

    if self.is_drawing:
        self._draw_segment(new_len, level - 1)
        if self.is_drawing:
            self.turtle_obj.left(90)
            self._draw_segment(new_len, level - 1)

        if self.is_drawing:
            self.turtle_obj.right(90)
            self._draw_segment(new_len, level - 1)

        if self.is_drawing:
            self.turtle_obj.right(90)
            self._draw_segment(new_len, level - 1)

        if self.is_drawing:
            self._draw_segment(new_len, level - 1)

        if self.is_drawing:
            self.turtle_obj.left(90)
            self._draw_segment(new_len, level - 1)

        if self.is_drawing:
            self.turtle_obj.left(90)
            self._draw_segment(new_len, level - 1)

        if self.is_drawing:
            self.turtle_obj.right(90)
            self._draw_segment(new_len, level - 1)

def draw_fractal(self, root, progress_callback, info_callback):
    self.close_window()

```

```

level = self.params.level.get()
side_length = self.params.length.get()

self.turtle_window = tk.Toplevel(root)
self.turtle_window.title(f"Minkowski Island (level {level})")
self.turtle_window.geometry("700x700")
self.turtle_window.protocol("WM_DELETE_WINDOW", self.close_window)

self.canvas = tk.Canvas(self.turtle_window, bg=self.params.bg_color,
highlightthickness=0)
self.canvas.pack(fill=tk.BOTH, expand=True)

self.turtle_window.update()

self.turtle_obj = turtle.RawTurtle(self.canvas)
self.turtle_obj.speed(self.params.speed.get())
self.turtle_obj.pencolor(self.params.color)
self.turtle_obj.pensize(self.params.pen_size.get())
self.turtle_obj.penup()

canvas_width = self.canvas.winfo_width()
canvas_height = self.canvas.winfo_height()

self.turtle_obj.screen.setworldcoordinates(0, canvas_height, canvas_width, 0)

if level == 0:
    fractal_size = side_length
else:
    fractal_size = side_length * (1.5**level)

start_x = (canvas_width - fractal_size) / 2
start_y = (canvas_height - fractal_size) / 2

self.turtle_obj.goto(start_x, start_y)

info_text = f"Level: {level} | Side length: {side_length}px | " f"Size:
{fractal_size:.0f}px"
self.canvas.create_text(canvas_width // 2, 20, text=info_text, fill="gray",
font=("Arial", 10))

self.turtle_obj.pendown()
self.is_drawing = True
progress_callback(f"Drawing level {level}...")
root.update()

try:
    for side in range(4):

```

```

        if not self.is_drawing:
            break
        self._draw_segment(side_length, level)
        if not self.is_drawing:
            break
        self.turtle_obj.right(90)
        progress_callback(f"Side {side + 1}/4")
        root.update()

        self.turtle_obj.hideturtle()
        progress_callback("Fractal ready")
        info_callback(level, side_length)

    except (RuntimeError, AttributeError):
        progress_callback("Error during drawing")

    finally:
        self.is_drawing = False

```

```

class MinkowskiIslandApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Fractal 'Minkowski Island'")
        self.root.geometry("400x700")
        self.root.resizable(False, False)

        self.params = FractalParams()
        self.renderer = FractalRenderer(self.params)
        self.info_text = None
        self.progress_var = None
        self.color_preview = None

        self._setup_ui()

    def _setup_ui(self):
        main_frame = ttk.Frame(self.root, padding=10)
        main_frame.pack(fill=tk.BOTH, expand=True)

        title = ttk.Label(main_frame, text="Minkowski Island", font=("Arial", 18,
"bold"))
        title.pack(pady=(0, 15))

        controls = UIControls(main_frame, self.params)

        controls.create_level_control()
        controls.create_length_control()
        controls.create_speed_control()

```

```

controls.create_pen_control()
controls.create_color_controls(self._choose_color, self._choose_bg_color)

self.color_preview = controls.color_preview

self._add_buttons(main_frame)
self._add_info_panel(main_frame)

self.progress_var = tk.StringVar(value="Ready")
progress_label = ttk.Label(main_frame, textvariable=self.progress_var,
font=("Arial", 9), foreground="green")
progress_label.pack(pady=(5, 0))

hint = ttk.Label(
    main_frame, text="Hint: start with level 1 and length 150", font=("Arial",
8), foreground="blue"
)
hint.pack(pady=(5, 0))

def _add_buttons(self, parent):
    btn_frame = ttk.Frame(parent)
    btn_frame.pack(fill=tk.X, pady=(10, 5))

    ttk.Button(btn_frame, text="DRAW FRACTAL",
command=self.draw_fractal).pack(fill=tk.X, pady=2)

    ttk.Button(btn_frame, text="TEST (square)",
command=self._test_drawing).pack(fill=tk.X, pady=2)

    ttk.Button(btn_frame, text="CLOSE WINDOW",
command=self._close_window).pack(fill=tk.X, pady=2)

def _add_info_panel(self, parent):
    info_frame = ttk.LabelFrame(parent, text="Information", padding=5)
    info_frame.pack(fill=tk.BOTH, expand=True, pady=(10, 0))

    self.info_text = tk.Text(info_frame, height=10, width=40, font=("Courier", 9))
    self.info_text.pack(fill=tk.BOTH, expand=True)
    self._update_info(1, 150)

def _choose_color(self):
    color = colorchooser.askcolor(title="Choose line color",
color=self.params.color)
    if color[1]:
        self.params.color = color[1]
        self.color_preview.config(bg=self.params.color)

def _choose_bg_color(self):

```

```

        color = colorchooser.askcolor(title="Choose background color",
color=self.params.bg_color)
        if color[1]:
            self.params.bg_color = color[1]
            if self.renderer.canvas and self.renderer.canvas.winfo_exists():
                self.renderer.canvas.config(bg=self.params.bg_color)

def _close_window(self):
    self.renderer.close_window()
    self.progress_var.set("Window closed")

def _test_drawing(self):
    self.renderer.test_drawing(self.root, self._update_progress)

def draw_fractal(self):
    self.renderer.draw_fractal(self.root, self._update_progress, self._update_info)

def _update_progress(self, message):
    self.progress_var.set(message)
    self.root.update()

def _update_info(self, level, length):
    self.info_text.config(state=tk.NORMAL)
    self.info_text.delete("1.0", tk.END)

    if level == 0:
        segments = 4
        perimeter = length * 4
        fractal_size = length
    else:
        segments = 4 * (8**level)
        perimeter = length * 4 * ((4 / 3) ** level)
        fractal_size = length * (1.5**level)

    info = (
        f"MINKOWSKI ISLAND\n"
        f"{'=' * 35}\n"
        f"Level: {level}\n"
        f"Side length: {length} px\n"
        f"Size: {fractal_size:.0f} px\n"
        f"Pen size: {self.params.pen_size.get()}\n"
        f"\nStatistics:\n"
        f"Segments: {segments:,}\n"
        f"Perimeter: {perimeter:.0f} px\n"
        f"\nDimension:\n"
        f"D = log(8)/log(4)\n"
        f"D = {math.log(8) / math.log(4):.3f}\n"
        f"\nFractal is a curve\n"

```

```

        f"of infinite length\n"
        f"in a bounded area"
    )

    self.info_text.insert("1.0", info)
    self.info_text.config(state=tk.DISABLED)

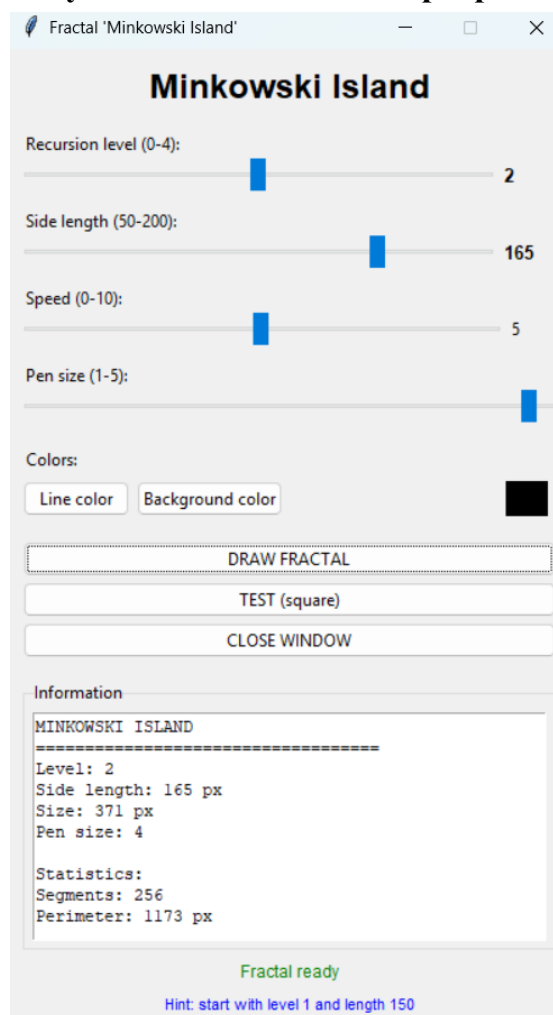
def run(self):
    self.root.mainloop()

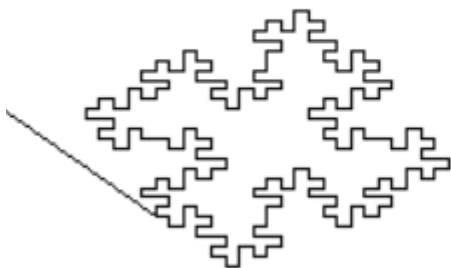
def main():
    root = tk.Tk()
    app = MinkowskiIslandApp(root)
    app.run()

if __name__ == "__main__":
    main()

```

Рисунки с выполнением программы





Level: 2 | Side length: 165px | Size: 371px

Вывод: освоил возможности языка программирования Python в разработке оконных приложений